

Costruttore Telescopico

- Consideriamo una classe che ha tanti attributi il cui valore una volta fissato non deve poter cambiare (ovvero attributi `final`)
- I valori di alcuni attributi potrebbero non essere necessari
- Si potrebbe usare una classe con costruttore telescopico, in modo da poter fornire solo i parametri necessari
- Quindi un costruttore con un parametro, uno con due parametri, uno con tre parametri, etc.
- Svantaggio: non è facile scrivere i client che passano così tanti parametri in modo corretto, poiché lato chiamante bisogna scoprire qual è l'ordine dei parametri

1

Prof. Tramontana - 26 Marzo 2025

```
public class Biglietto {
    private final String titolare, agente;
    private final LocalDateTime data;
    private final double prezzo;
    private double acconto;

    public Biglietto(String t, String a, LocalDateTime d, double p) {
        data = d;
        titolare = t;
        prezzo = p;
        agente = a;
    }
    public Biglietto(String t, String a, LocalDateTime d) {
        this(t, a, d, 100);
    }
    public Biglietto(String t, String a) {
        this(t, a, LocalDateTime.now());
    }
    public Biglietto(String t) {
        this(t, "BBBSelling"); // uso valori prestabiliti se non sono passati
    }

    // altri metodi utili (e altre combinazioni di parametri)
}
```

2

Prof. Tramontana - 26 Marzo 2025

Metodi Set Separati

- L'alternativa è predisporre metodi set, per assegnare gli attributi mano a mano (quindi gli attributi **non sono** `final`)
- Svantaggi
 - (i) non si possono avere attributi immutabili
 - (ii) non si sa se impostando i valori singolarmente si è arrivati a uno stato consistente e il controllo sulla consistenza non è possibile nel costruttore

3

Prof. Tramontana - 26 Marzo 2025

```
public class Biglietto {
    private String titolare, agente;
    private LocalDateTime data;
    private double prezzo;

    private double acconto;

    public Biglietto() {
    }
    public void setPrezzo(double p) {
        prezzo = p;
    }
    public void setData(LocalDateTime d) {
        data = d;
    }
    public void setAgente(String a) {
        agente = a;
    }
    public void setTitolare(String t) {
        titolare = t;
    }

    // altri metodi utili
}
```

4

Prof. Tramontana - 26 Marzo 2025

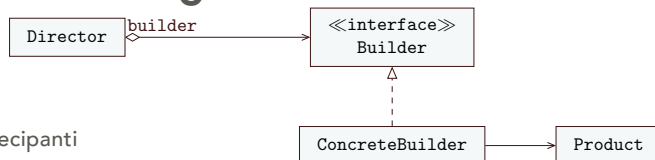
Soluzione

- Soluzione: richiedere l'oggetto desiderato indirettamente, attraverso il *design pattern Builder*
- Passi:
 1. Il client istanzia la classe *Builder*, passando al costruttore i parametri richiesti, quindi al client viene restituito un oggetto di tipo *Builder*
 2. Il client chiama i metodi set sul *Builder* per ciascun attributo nell'oggetto desiderato
 3. Infine, il client chiama un metodo (*build*) che genera l'oggetto desiderato che è immutabile. Il metodo *build* può controllare se i parametri necessari sono presenti (verifica se lo stato è consistente prima di creare l'istanza)

5

Prof. Tramontana - 26 Marzo 2025

Design Pattern Builder



- Partecipanti
 - **Builder** specifica un'interfaccia per creare parti di un Product
 - **ConcreteBuilder** (*BigliettoBuilder*) costruisce ed assembla parti del Product, implementando l'interfaccia Builder; definisce e tiene traccia della rappresentazione che crea; fornisce un'interfaccia per ottenere il Product. ConcreteBuilder costruisce la rappresentazione interna del Product e definisce il processo con cui è assemblato.
 - **Director** (*InitBiglietto*) costruisce un oggetto Product usando l'interfaccia Builder
 - **Product** (*Biglietto*) rappresenta l'oggetto complesso da costruire. Product definisce le parti costituenti che vengono assemblate nel risultato finale

7

Prof. Tramontana - 26 Marzo 2025

Design Pattern Builder

- **Intento:** separare la costruzione di un oggetto complesso dalla sua rappresentazione così che lo stesso processo costruttivo può creare diverse rappresentazioni
- Applicabilità
 - Usare il pattern Builder quando l'algoritmo per creare un oggetto complesso dovrebbe essere indipendente dalle parti che compongono l'oggetto e da come sono assemblate
 - Il processo di costruzione deve permettere diverse rappresentazioni dell'oggetto che è costruito

6

Prof. Tramontana - 26 Marzo 2025

```
public class Biglietto { // Product
    private final String titolare, agente;
    private final LocalDateTime data;
    private final double prezzo;
    private double sconto;
    public Biglietto(String t, String a, double p, LocalDateTime d) {
        data = d;
        titolare = t;
        prezzo = p;
        agente = a;
    }
    // altri metodi utili
}

public class BigliettoBuilder implements BBuilder { // ConcreteBuilder
    private String titolare, agente = "SingleSeller";
    private LocalDateTime data;
    private double prezzo = 50;
    public BigliettoBuilder agente(String a) {
        agente = agente + " " + a;
        return this;
    }
    public BigliettoBuilder titolare(String t) {
        titolare = t;
        return this;
    }
    public BigliettoBuilder sconto(double sc) {
        if (prezzo >= 40 && sc <= 10)
            prezzo = (1 - sc / 100) * prezzo;
        return this;
    }
    public Biglietto getBiglietto() { // metodo build
        if (titolare == null) return null;
        return new Biglietto(titolare, agente, prezzo, data);
    }
    // altri metodi utili
}

public class InitBiglietto { // Director
    public Biglietto getBiglietto() {
        Biglietto b = new BigliettoBuilder()
            .agente("Tim Seller")
            .titolare("Mario Rossi")
            .sconto(5)
            .data(LocalDateTime.now())
            .getBiglietto();
        return b;
    }
}
```

8

Prof. Tramontana - 26 Marzo 2025

Builder

- Conseguenze
 - La rappresentazione interna del Product può cambiare. Il Builder nasconde la rappresentazione e la struttura interna del Product. Per cambiare la rappresentazione interna del Product si può definire un nuovo tipo di Builder
 - Isola il codice di costruzione da quello di rappresentazione. Migliora la modularità incapsulando il modo in cui un oggetto complesso è costruito e rappresentato. I client non devono conoscere alcunché sulle classi che definiscono la struttura interna del Product. Ogni ConcreteBuilder contiene tutto il codice per creare e assemblare un particolare tipo di Product
 - Fornisce un controllo fine sul processo di costruzione, in quanto il Product è costruito passo dopo passo dal Director. Solo quando il prodotto è completo viene preso da dentro il Builder